Dia 1 — Como o Lean confere uma demonstração

Folha de apoio e exercícios para casa

Hoje em uma frase

Hoje em uma frase

Existe uma forma de checar mecanicamente se uma prova está correta?

Como saber se uma demonstração é correta?

Existe uma forma de **checar mecanicamente** se uma demonstração está correta? Há duas respostas, e elas se complementam.

Resposta 1: revisão por pares

Na **revisão por pares**, especialistas da área (os *pares*) leem o trabalho e julgam se ele está correto antes da publicação. É o que fazemos há muito tempo, e funciona bem, mas tem limites: o revisor confere as ideias, não cada passo; tem tempo e atenção finitos; e aceita "é fácil ver que...". Erros sobrevivem. A "demonstração" de Kempe (1879) do teorema das quatro cores foi aceita por 11 anos, até Heawood achar a falha, em 1890.

Resposta 2: assistentes de prova

Um **assistente de prova** é um programa que confere, passo a passo, se uma demonstração está correta *em cada passo lógico*, sem aceitar lacunas. Exemplos: **Lean**, Rocq (antigo Coq) e Isabelle. O objetivo é garantir validade, completude e ausência de lacunas. Não se trata de substituir o matemático, e sim de dar a ele um revisor que não cansa.

	Quem constrói a prova	**Quem confere**
Revisão por pares	o autor	outros matemáticos, lendo
Provador automático (SAT/SMT)	a máquina, sozinha, numa lógica restrita	a própria máquina
Assistente de prova (Lean)	você ou uma IA (no Lean)	o **kernel** do Lean, a cada passo

O que é o Lean

O **Lean 4** é, ao mesmo tempo, uma **linguagem de programação funcional** e um **assistente de prova**: nele você escreve definições, funções e demonstrações, e o computador confere tudo.

Criado em 2013 pelo brasileiro Leonardo de Moura (doutor pela PUC-Rio) na Microsoft Research, é um projeto de código aberto, mantido pela *Lean FRO*, uma organização sem fins lucrativos. Em torno dele há uma grande comunidade de matemáticos e a **Mathlib**, uma uma biblioteca aberta e colaborativa de matemática formalizada desenvolvida pela comunidade com aproximadamente 290.000 teoremas o lemas de matemática verificada.

O que o Lean tem a ver com a IA?

Mito

Se a IA escreveu uma demonstração "convincente", então o resultado está provado!

Realidade

Os modelos LLM trabalham com probabilidade, logo podem alucinar. Estes modelos conseguem apontar possíveis falhas em demonstrações, mas não conseguem garantir se uma demonstração possui efetivamente um erro. O Lean, por sua vez, dá um veredito binário: a prova é aceita ou recusada.

Os grandes modelos de linguagem (ChatGPT, Claude, Gemini, ...) produzem texto *provável*, não texto *verificado*: uma demonstração pode soar convincente e ainda assim ter um passo errado. Quando a IA escreve o raciocínio em **Lean**, o código passa pelo kernel, e o veredito deixa de depender de quem escreveu.

Isso também explica por que provas em Lean são ótimos dados de treinamento para a IA: cada passo já foi verificado mecanicamente, então o modelo aprende com exemplos que sabidamente estão certos.

Exemplo: AlphaProof (DeepMind, 2024). Resolveu 4 dos 6 problemas da Olimpíada Internacional de Matemática de 2024. Os enunciados foram traduzidos por humanos para o Lean; a IA buscava provas, e *o Lean decidia* se estavam certas, sem ambiguidade.

O que o Lean *não* faz

- **Não** decide se uma afirmação é tautologia (verdadeira em todas as atribuições) ou satisfazível (verdadeira em alguma). Ele não trabalha com tabelas-verdade: não há uma estrutura onde variáveis recebem valores e fórmulas são avaliadas.
- **Não** verifica afirmações pela semântica da lógica de primeira ordem ("verdade em todos os modelos"), nem procura demonstrações sozinho num sistema dedutivo.
- **Não** é um verificador SAT/SMT (*Satisfatibilidade Módulo Teorias*), como Z3 ou CVC5, que recebem um enunciado e decidem sozinhos se ele pode ser verdadeiro.

Por quê? Porque as proposições do Lean não são fórmulas de uma lógica pequena e decidível. A lógica do Lean é muito mais expressiva: fala de funções, conjuntos, estruturas algébricas, espaços topológicos... e, nela, decidir a verdade de um enunciado é impossível em geral. Por isso a divisão de trabalho é outra: **você constrói a prova, o Lean confere**. Algumas táticas automatizam pedaços (`decide`, `omega`, `simp`), mas elas também *produzem* uma prova, que o kernel confere do mesmo jeito.

O Lean não é um "SAT solver". Um provador automático (Z3, CVC5, Vampire) recebe um enunciado e *procura sozinho* uma prova ou um contraexemplo, em lógicas onde isso é possível. O Lean recebe um enunciado *e uma prova construída por você* e *confere* se a prova tem o tipo certo. Algumas táticas do Lean automatizam pedaços (como `simp` ou `omega`), mas elas também produzem uma prova, que o kernel confere igual.

SAT/SMT e Lean: dois problemas diferentes

- **SAT/SMT:** você dá uma proposição, e a máquina verifica *automaticamente* se ela pode ser verdadeira, sem nenhuma ajuda humana.

- **Lean:** verifica se um *objeto de prova construído pelo usuário* tem o tipo correto. É um problema fundamentalmente diferente.

A ideia central: enunciados são tipos, demonstrações são objetos

O Lean não "descobre" demonstrações sozinho e não testa milhões de casos. Ele é um **conferente**: você escreve a demonstração, e um programa pequeno e muito vigiado (o **kernel**) verifica se cada passo segue as regras. Se sobrar alguma lacuna, ele recusa e diz onde. Quando não sobra nada a provar, a demonstração está aceita — sem "quase certo".

O que o Lean faz: teoria de tipos dependentes

Mito

Verificação formal é uma caixa-preta: não dá para saber o que a máquina checou.

Realidade

Um programa pequeno e isolado, o *kernel*, checa uma única coisa: se o termo t tem o tipo P. Provar é construir; verificar é checar o tipo.

O Lean verifica demonstrações usando a **teoria de tipos dependentes**. A Teoria de tipos é uma subárea da lógica que estuda sistemas formais que classificam expressões ou objetos matemáticos através de seus tipos. Na teoria de tipos, todo objeto tem um **tipo**: uma coleção de objetos de mesma natureza, com suas regras e operações (pense em $x \in \mathbb{N}$, que no Lean se escreve `x : ℕ`). A Teoria de tipos dependentes é uma extensão da teoria de tipos em que um tipo é **dependente** quando depende de um valor.

Exemplos:

- `Fin n`, o conjunto $\{0, 1, \ldots, n-1\}$, é um tipo diferente para cada n (assim como $\mathbb{R}^n$ depende de n);
- o enunciado `n + 0 = n` depende de `n`. Uma prova de `∀ n : ℕ, n + 0 = n` é uma *função* que, para cada `n`, devolve uma prova de `n + 0 = n`: o tipo do resultado depende da entrada.

```
#check Fin 3                          -- Fin 3 : Type
#check (∀ n : ℕ, n + 0 = n)           -- ∀ (n : ℕ), n + 0 = n : Prop
#check fun n : ℕ => Nat.add_zero n    -- ... : ∀ (n : ℕ), n + 0 = n
```

A última linha mostra, na tela do Lean, que uma *função* tem como tipo um enunciado com $\forall$. É a ideia da próxima seção.

Curry–Howard: enunciados são tipos, demonstrações são objetos

A grande sacada, a **correspondência (isomorfismo) de Curry–Howard**, é tratar os *enunciados* como tipos e as *demonstrações* como objetos desses tipos:

Na matemática	No Lean	Leia como
proposição P	um tipo `P : Prop`	"P é um enunciado"
demonstração de P	um termo `t : P`	"t é uma prova de P"
$P \Rightarrow Q$	função `P → Q`	transforma prova de P em prova de Q
$P \wedge Q$	par `⟨hp, hq⟩`	uma prova de P *e* uma de Q
$P \vee Q$	`Or.inl hp` ou `Or.inr hq`	uma prova de P *ou* uma de Q, dizendo qual
$\forall x,\ P(x)$	função `fun x => ...`	para cada x, uma prova de $P(x)$ (tipo dependente)
$\exists x,\ P(x)$	par `⟨a, ha⟩`	uma testemunha a e uma prova de $P(a)$

Universos do Lean e a Correspondência de Curry-Howard

O Lean possui dois universos principais:

- **`Prop`** (Universo das Proposições): os elementos em `Prop` são as provas de proposições lógicas. Exemplos de proposições lógicas: $2 + 2 = 4$, $\forall n,\ n + 0 = n$.

  Obs.: Proposições em `Prop` são apagadas durante a compilação. Nesse sentido proposições não geram código.

- **`Type`** (Universo dos Tipos): os elementos dos tipos são dados computacionais, estruturas ou valores.
  Exemplos de tipos: `Nat`, `List Nat`.

Todo valor no Lean pertence a algum tipo. Pode ser `Nat`, `Int`, `String`, `Bool`, etc.

Para um tipo em `Type`, o que é o valor importa (por exemplo 3, 4, 5 são valores distintos em `Nat`).

O que o Lean faz quando você escreve uma prova?

1. Você escreve um **termo de prova** t e declara `theorem meu_lema : P := t`.
2. O **kernel** confere se `t : P` é válido segundo as regras da teoria de tipos dependentes.
3. Se a verificação passa, a prova está **correta**: não há grau de certeza nem heurística.
4. Se falha, o Lean reporta um erro de tipo, como um compilador.

O kernel é um programa pequeno, simples, isolado e extremamente protegido. Ele não precisa "entender" a matemática: só precisa checar tipos. É por isso que podemos confiar nele.

```
theorem meu_teorema : 2 + 2 = 4 := by rfl
```

Pedaço	Nome	O que é
`meu_teorema`	nome	Como vamos citar este resultado depois.
`2 + 2 = 4`	enunciado	Um tipo em `Prop`. É o tipo de `meu_teorema`.
`by rfl`	demonstração	"É imediato, por definição": o kernel calcula os dois lados e vê que coincidem.

Em matemática escrevemos $x \in \mathbb{N}$ e significa "x é um natural". No Lean escreve `x : ℕ` e o computador lê "x tem tipo $\mathbb{N}$". Um **tipo** é, para nós, uma coleção de objetos de mesma natureza. O coração do Lean então está na Correspondência de **Curry–Howard** (ou Isomorfismo de Curry-Howard), nomeada em homenagem aos matemáticos Haskell Curry e William Howard. Este princípio estabelece uma equivalência estrutural entre programas de computador e provas matemáticas: proposições correspondem a tipos, e provas correspondem a termos que habitam esses tipos.

Na matemática	No Lean	Leia como
proposição P	um tipo `P : Prop`	"P é um enunciado"
demonstração de P	um termo `t : P`	"t é uma prova de P"
$P \Rightarrow Q$	função `P → Q`	transforma prova de P em prova de Q
$P \wedge Q$	par `⟨hp, hq⟩`	uma prova de P *e* uma de Q
$\forall x,\ P(x)$	função `fun x => ...`	para cada x, uma prova de $P(x)$

Por isso há **dois universos**: `Prop`, onde moram os enunciados (`2 + 2 = 4`, `∀ n, n + 0 = n`), e `Type`, onde moram os dados (ℕ, listas, funções que calculam).

Demonstrar é construir um objeto do tipo certo; o kernel confere o tipo.

Exemplo dos slides, que você pode digitar no Lean Web:

```
theorem meu_teorema : 2 + 2 = 4 := by rfl
```

Pedaço	Nome	O que é
`meu_teorema`	nome	Como vamos citar este resultado depois.
`2 + 2 = 4`	enunciado	Um tipo em `Prop`. É o tipo de `meu_teorema`.
`by rfl`	demonstração	"É imediato, por definição": o kernel calcula os dois lados e vê que coincidem.

IA, Lean e pesquisa matemática

Mito

Isso é brinquedo: serve para $2 + 2 = 4$, não para matemática de verdade.

Realidade

IA e Lean já estão sendo usados em problemas de pesquisa, e é o kernel do Lean que torna esses resultados confiáveis.

Dois exemplos recentes, apresentados nos slides (segundo os anúncios das próprias empresas; os links estão no repositório do minicurso):

- **OpenAI (outubro de 2026):** publicou 722 artigos gerados por um modelo de IA interno, alegando ter resolvido ou avançado em cerca de 372 problemas de 17 áreas da matemática pura. Em vez de confiar apenas no texto gerado, as demonstrações foram traduzidas para o Lean.
- **Axiom Math (AxiomProver):** startup de IA fundada em 2025, voltada ao raciocínio matemático e à eliminação de "alucinações". Em 2026, anunciou ter formalizado em Lean as demonstrações do *Stacks Project* (o grande livro colaborativo de geometria algébrica) e a classificação dos grupos simples finitos, uma demonstração de mais de 10.000 páginas espalhada por centenas de artigos.

Repare no padrão: a IA *propõe*, o Lean *certifica*. Sem o verificador, seriam milhares de páginas que nenhum ser humano conseguiria revisar.

Mãos no teclado: o Lean Web

Preparação

Abra `live.lean-lang.org` no navegador: não é preciso instalar nada, e o Lean 4 com a Mathlib roda no servidor. Apague o que estiver lá e cole o cabeçalho da aula na primeira linha (a primeira carga demora um pouco):

```
import Mathlib.Data.Nat.Basic
```

Depois digite, na linha 3, o primeiro comando. No painel da direita (**Infoview**), em *Messages*, deve aparecer 7:

```
#eval 2 + 5
```

O Lean Web conversa com o Lean pelo **LSP** (*Language Server Protocol*): a cada tecla, o servidor confere o que você escreveu e atualiza o Infoview. É por isso que você vê o estado da prova "em tempo real".

Programas são expressões matemáticas

No Lean, funções são escritas como no λ-cálculo: $\lambda x.\, x+1$ vira `fun x => x + 1`, e a função $f\colon \mathbb{N} \to \mathbb{N}$, $f(n) = n+1$, vira uma definição. Os programas não têm efeitos colaterais: toda expressão tem um valor, como uma função matemática.

```
#eval (fun x => x + 1) 3              -- 4
def f : Nat → Nat := fun n => n + 1
#eval f 41                            -- 42
```

Declarações fundamentais

```
def dobro (n : Nat) : Nat := 2 * n    -- define uma função (um dado, em Type)
#check dobro                          -- dobro (n : Nat) : Nat
#check dobro 5                        -- dobro 5 : Nat
#eval dobro 5                         -- 10
theorem dobro_par (n : Nat) : dobro n % 2 = 0 := by
  simp [dobro]                        -- uma prova, em modo tática
```

- `def` define objetos e funções; `theorem` (ou `example`, que é um teorema sem nome) define provas.
- `#check` responde "qual o tipo disto?"; `#eval` responde "quanto vale isto?". As respostas aparecem no Infoview.
- `by` abre o modo tática; sem `by`, você escreve o termo de prova diretamente.
- `dobro (n : Nat) : Nat` é o jeito do Lean escrever dobro$\colon \mathbb{N} \to \mathbb{N}$.

O exemplo do kernel, agora digitado

Digite no Lean Web o exemplo que vimos na parte teórica:

```
theorem meu_teorema : 2 + 2 = 4 := by rfl
```

Pedaço	Nome	O que é
`meu_teorema`	nome	Como vamos citar este resultado depois.
`2 + 2 = 4`	enunciado	Um tipo em `Prop`. É o tipo de `meu_teorema`.
`by rfl`	demonstração	"É imediato, por definição": o kernel calcula os dois lados e vê que coincidem.

Estado da prova: o painel mais importante

Coloque o cursor logo depois do `by`. O painel da direita (**Infoview**) mostra o **estado da prova**:

```
example (n : \Nat) : n + 0 = n := by
  -- cursor aqui
```

Linha	Nome	O que é
`n : ℕ`	contexto	O que temos à mão: objetos e hipóteses. Aqui, só "seja n natural".
`⊢ n + 0 = n`	meta	O que falta provar. O símbolo ⊢ quer dizer "falta mostrar que".

Analogia: a demonstração no quadro

- O **contexto** é a parte "Sejam... e suponha que..." do enunciado: o que o problema nos dá.
- A **meta** é a tese escrita no canto do quadro, esperando ser demonstrada.
- Uma **tática** é um passo justificado da demonstração ("por definição", "aplicando o lema...").
- O **Infoview** é o quadro atualizado depois de cada passo: o que já temos e o que ainda falta.
- Quando aparece **No goals** ("sem metas"), é o *c.q.d.*

Um passo, visto de perto: modus ponens

Temos P, temos $P \Rightarrow Q$, queremos Q. No quadro: "para provar Q, basta provar P (pela implicação); e P é hipótese". No Lean, em **modo tática**:

```
example (P Q : Prop) (hp : P) (hpq : P → Q) : Q := by
  apply hpq      -- antes:  ⊢ Q     depois:  ⊢ P
  exact hp       -- antes:  ⊢ P     depois:  No goals
```

Tela real antes e depois do `apply hpq` (o contexto não muda; só a meta):

```
P Q : Prop                          P Q : Prop
hp : P                              hp : P
hpq : P → Q                         hpq : P → Q
⊢ Q                                 ⊢ P
```

A mesma demonstração em **modo termo**, escrita de uma vez: `hpq hp`. Como `hpq` é uma função de provas de P em provas de Q, aplicá-la a `hp` dá uma prova de Q:

```
theorem meu_mp (P Q : Prop) (hp : P) (hpq : P → Q) : Q := hpq hp
```

Termo × tática: o termo é o objeto pronto; as táticas são instruções para construí-lo passo a passo, olhando a meta. Os dois viram a mesma coisa para o kernel. Nos exercícios, usaremos mais táticas, porque se parecem com a escrita matemática usual.

Lema × tática: o resultado citado e o ato de citá-lo

`hpq`, `Nat.add_zero` ("$n+0=n$"), `Nat.add_comm` são **lemas** (ou hipóteses): fatos já provados, como o teorema do livro que você cita. `apply`, `exact`, `rw` e `rfl` são **táticas**: o ato de usar um fato. Na linha `apply hpq`, o fato é `hpq` e o ato é `apply`.

Quando o Lean recusa

Se, no exemplo acima, você escrever `exact hpq` em vez de `apply hpq`, a linha fica sublinhada em vermelho e aparece:

```
Type mismatch
  hpq
has type
  P → Q
but is expected to have type
  Q
```

Leia como um colega atento: "você me entregou uma prova de $P \Rightarrow Q$, mas a tese é Q". Não é punição; é informação sobre *qual* passo não se justifica. A meta continua lá, esperando outro passo. Note que é exatamente Curry–Howard em ação: o erro é de *tipo*.

Dica:

- Coloque o cursor *dentro* de um bloco `by` para ver o estado da prova.
- Se aparecer **No goals** em verde, a prova está completa!

Ecossistema: Mathlib e onde encontrar o Lean

A **Mathlib** é a biblioteca de matemática unificada da comunidade Lean: centenas de milhares de definições e teoremas já verificados, de álgebra (grupos, anéis, módulos, corpos) a topologia, análise, teoria dos números, geometria e combinatória, mantida por centenas de colaboradores. Você não recomeça do zero: experimente no Lean Web

```
example (a b : Nat) : a + b = b + a := by exact?
```

e o Lean procura sozinho o lema que fecha a meta (`Try this: exact Nat.add_comm a b`).

Onde encontrar o Lean

Onde	Para quê
`Repositório do minicurso`	Slides, folhas e códigos do minicurso.
`live.lean-lang.org`	Lean Web: o Lean com a Mathlib no navegador, sem instalar nada.
`adam.math.hhu.de`	*Natural Number Game* e outros jogos (base do Dia 2).
`lean-lang.org`	Página oficial: instalação no seu computador (VS Code + extensão Lean 4) e a seção *Learn*.
Mathematics in Lean	O livro para matemáticos, já usando a Mathlib.
Theorem Proving in Lean 4	A lógica e as táticas do Lean, com calma.
Functional Programming in Lean	O Lean como linguagem de programação.
`mathlib4_docs`	Documentação da Mathlib, para procurar lemas.
`loogle.lean-lang.org`	Busca de lemas por padrão, por exemplo `?a + ?b = ?b + ?a`.
`reservoir.lean-lang.org`	Catálogo de pacotes do Lean (a Mathlib está lá).
`leanprover.zulipchat.com`	Comunidade (canal #new members): muito receptiva a iniciantes.

Resumo do Dia 1

1. **Motivação:** provas matemáticas podem conter erros; um assistente de prova confere mecanicamente *cada passo* lógico.
2. **IA e Lean:** os LLMs trabalham com probabilidades e podem alucinar; o Lean dá um veredito binário: a prova é aceita ou recusada.
3. **O que o Lean não faz:** não testa casos nem decide sozinho se um enunciado é verdadeiro. Isso é o que fazem os provadores SAT/SMT.
4. **O que o Lean faz:** trabalha com teoria de tipos dependentes. Pela correspondência de Curry–Howard, proposições são tipos e provas são termos; o kernel confere se `t : P`.
5. **Pesquisa:** IA e Lean já estão sendo usados em problemas de pesquisa em matemática, e o kernel é o que torna esses resultados confiáveis.
6. **Prática:** `live.lean-lang.org`, `#eval`, `def`, `theorem`, termos $\times$ táticas e o estado da prova (Infoview).
7. **Ecossistema:** a Mathlib e onde continuar estudando.

Hoje em uma frase

Provar é construir um objeto do tipo certo. O Lean confere; você guia.

Glossário (Dia 1)

Palavra	Em uma linha — e o análogo no papel
Revisão por pares	Especialistas leem e julgam um trabalho antes da publicação. *O parecer do referee.*
Assistente de prova	Programa que confere demonstrações escritas por você. *O revisor incansável.*
SAT/SMT	Provadores automáticos que decidem sozinhos, em lógicas restritas, se um enunciado pode ser verdadeiro. *Um algoritmo de decisão.*
Teoria de tipos dependentes	A base lógica do Lean: todo objeto tem tipo, e tipos podem depender de valores (`Fin n`). *Os "fundamentos" do Lean, no lugar de ZFC.*
Kernel	O núcleo pequeno do Lean que confere se `t : P`. *O árbitro final.*
Tipo	Coleção de objetos de mesma natureza ($\mathbb{N}$, $\mathbb{N} \to \mathbb{N}$). *O "$\in$" de $x \in \mathbb{N}$.*
`Prop` / `Type`	Universo dos enunciados / universo dos dados. *Afirmações × objetos.*
Termo	Um objeto de algum tipo; quando o tipo é um enunciado, é uma prova. *A demonstração pronta.*
Curry–Howard	Enunciado = tipo; prova = termo; $\Rightarrow$ = função. *Provar é construir.*
`def`	Define um objeto ou função. *"Definimos $f(n) = 2n$".*
`theorem` / `example`	Enunciado + prova (com nome / sem nome). *Teorema do livro / exercício resolvido.*
`#check`	Mostra o tipo de uma expressão. *"Que tipo de objeto é este?"*
`#eval`	Calcula o valor de uma expressão. *Fazer a conta.*
`fun x => ...`	Função anônima (λ). $x \mapsto x + 1$.
`by`	Entra no modo tática. *"Demonstração:".*
Infoview	Painel que mostra o estado da prova. *O quadro atualizado.*
Contexto / hipótese	O que está acima do $\vdash$, com nomes (`hp : P`). *"Sejam..., suponha que..."*
Meta ($\vdash$)	O que falta provar. *A tese.*
No goals	Nada mais a provar: prova aceita. *c.q.d.*
Tática	Um passo que transforma a meta. *Um passo justificado.*
Lema	Fato já provado, citado por nome (`Nat.add_zero`). *O resultado do livro.*
`rfl`	Fecha `a = b` quando os dois lados coincidem por cálculo. *"É imediato, por definição".*
`exact h`	Entrega a prova exata da meta. *"...que é a hipótese h."*
`apply h`	Com `h : P → Q` e meta `Q`, troca a meta por `P`. *"Basta provar P".*
`intro h`	Com meta `P → Q`, põe `h : P` no contexto. *"Suponha P".*
`rw [h]`	Troca, na meta, o lado esquerdo de `h` pelo direito. *Substituição.*
`rw [← h]`	O mesmo, no sentido contrário. *Substituir de trás para frente.*
$\langle$`hp, hq`$\rangle$, `h.1`, `h.2`	Monta / desmonta uma prova de `P ∧ Q`. *"P e Q"; "em particular, P".*
Mensagem de erro	O Lean recusou um passo e diz por quê. *O comentário do revisor.*
Mathlib	Biblioteca de matemática verificada do Lean. *A biblioteca da universidade.*

Como digitar os símbolos: $\mathbb{N}$ = `\N` $\to$ = `\to` $\leftarrow$ = `\l` $\wedge$ = `\and` $\langle\ \rangle$ = `\<` e `\>` (digite e aperte espaço). Você também pode escrever `Nat` no lugar de $\mathbb{N}$ e `->` no lugar de $\to$.

Exercícios para casa

Tempo estimado: **1h a 1h30**. Faça na ordem. Regra de ouro: **antes de apertar Enter, escreva o que você espera ver** (o tipo, o valor, a nova meta) e só depois confira. Acertar a previsão é o que mostra que você entendeu. As dicas estão no fim da folha: tente antes de olhar. O Exercício 4 é o mais importante: ele é o aquecimento do Dia 2.

Onde: Lean Web = `live.lean-lang.org`. Apague o que estiver lá e deixe o cabeçalho da aula na primeira linha (a primeira carga demora um pouco):

```
import Mathlib.Data.Nat.Basic
```

Digite o código em vez de copiar do PDF: a cópia costuma estragar espaços e aspas. **NNG** = Natural Number Game, em `adam.math.hhu.de`.

Exercício 1 — Prever tipo e valor (Lean Web)

Digite as duas definições abaixo. Depois, *antes* de digitar cada linha da lista, escreva no papel o que o Infoview vai mostrar; então digite e confira.

```
def dobro (n : Nat ) : Nat := 2 * n
theorem meu_teorema : 2 + 2 = 4 := by rfl
```

```
#check 2 + 3
#eval 2 + 3
#check dobro
#eval dobro 21
#check (2 + 2 = 4)
#check meu_teorema
#eval 7 - 10
#eval 7 / 2
```

Depois responda: (a) quais respostas são *enunciados* e quais são *provas*? (b) os dois últimos resultados surpreenderam? Por que o Lean faz assim? *Traga:* a tabela "previsto × obtido".

Exercício 2 — O `rfl` fecha ou não? (Lean Web)

Para cada enunciado, primeiro *aposte*: o `rfl` fecha a meta? Depois teste, digitando por exemplo `example : 2 + 2 = 4 := by rfl` (para os que têm `n`, escreva `example (n : ℕ) : n + 0 = n := by rfl`).

(a) `2 + 2 = 4` (b) `n + 0 = n` (c) `0 + n = n` (d) `2 * 3 = 6` (e) `2 + 2 = 5`

Para os que falharam, copie a mensagem de erro e explique em uma frase o que ela diz. Um deles falha por um motivo muito diferente dos outros: qual, e por quê? *Traga:* as apostas, os resultados e a sua explicação — vamos usá-la no começo do Dia 2.

Exercício 3 — Dois modos, a mesma prova (Lean Web)

Enunciado: se P, $P \Rightarrow Q$ e $Q \Rightarrow R$, então R.

(a) Escreva a prova no papel, em duas ou três frases.

(b) Complete em **modo tática** (troque `sorry` por táticas; antes de cada linha, escreva a meta que você espera ver):

```
example (P Q R : Prop) (hp : P) (hpq : P → Q) (hqr : Q → R) : R := by
  sorry
```

(c) Agora em **modo termo**: escreva, depois do `:=` (sem `by`), uma única expressão que seja uma prova de `R`.

Traga: as três versões, com as linhas do papel alinhadas às linhas do Lean.

Exercício 4 — Tutorial World inteiro (NNG) — aquecimento para o Dia 2

Abra o NNG e jogue o *Tutorial World* completo (8 níveis, de *The rfl tactic* até $2 + 2 = 4$). Em cada nível:

(a) copie no papel a tela inicial e marque **objetos**, **hipóteses** e **meta**;

(b) antes de cada tática, escreva a meta que você espera ver; depois confira.

No nível final ($2 + 2 = 4$), escreva a demonstração como texto de livro: um passo por linha, no formato *"Pelo lema _____, . . . vira . . ."*. *Traga:* as anotações do nível 2 e o texto do $2 + 2 = 4$. (No jogo, `succ n` quer dizer "o sucessor de n", isto é, $n + 1$. O jogo libera as táticas aos poucos: use só as que aparecem na lista ao lado.)

Exercício 5 — A IA propõe, o Lean julga (chat de IA + Lean Web)

(a) Peça a um chat de IA (ChatGPT, Claude, Gemini...): *"Escreva uma prova em Lean 4 de que, para todo natural n, $n + 0 = n$."* Cole a resposta no Lean Web (abaixo do `import Mathlib.Data.Nat.Basic`). O Lean aceitou? Se recusou, copie a mensagem de erro.

(b) Repita com *"para todo natural n, $0 + n = n$"*. Compare com o Exercício 2(c).

Traga: as provas propostas e o veredito do Lean para cada uma. Não se preocupe se não entender a prova de (b): ela é o assunto do Dia 2.

Exercício 6 — Desafio opcional

(a) Em no máximo 3 linhas, explique a um colega por que o Lean *não* é um "SAT solver" e por que isso não é um defeito.

(b) Prove que $P \wedge Q \Rightarrow Q \wedge P$, em modo tática *e* em modo termo:

```
example (P Q : Prop) : P ∧ Q → Q ∧ P := by
  sorry
```

Se não sair, tudo bem: traga até onde chegou.

Se travar

1. **Releia a meta** (a linha com $\vdash$) e o contexto acima dela. Pergunte: o que eu tenho, e o que falta?
2. **Olhe as hipóteses e os lemas disponíveis.** Alguma hipótese tem exatamente o formato da meta (`exact`)? Alguma é uma implicação que *termina* na meta (`apply`)?

3. **No NNG, use o botão de dicas** do próprio jogo.

4. **Pergunte a um chat de IA pedindo explicação, não a resposta.** Modelo de pergunta: *"Estou aprendendo Lean 4 no Lean Web. Minha meta é* [*cole aqui*] *e minhas hipóteses são* [*cole aqui*]*. Não me dê a solução: explique o que a tática apply faria aqui e me faça uma pergunta que me ajude a pensar."* Ou, para um erro: *"O Lean me deu esta mensagem:* [*cole aqui*]*. Explique em linguagem de matemático o que ela quer dizer, sem corrigir o código para mim."* (A IA pode errar detalhes, e às vezes responde em Lean 3, que é diferente; quem decide é o Lean.)

5. **Anote a dúvida** (com a meta que estava na tela) e traga para o Dia 2. Travar é informação, não fracasso.

Dicas graduadas

Leia só a Dica 1; tente de novo; só então passe para a próxima.

Exercício 1

Dica 1

`#check` devolve "expressão `:` tipo"; `#eval` devolve só um valor.

Dica 2

Em $\mathbb{N}$ não há números negativos, e `/` entre naturais é a divisão inteira (o quociente). O Lean precisa devolver *algum* natural em `7 - 10`.

Quase a resposta

`2 + 3` : $\mathbb{N}$, `5`, `dobro (n : ℕ) : ℕ`, `42`, `2 + 2 = 4 : Prop`, `meu_teorema : 2 + 2 = 4`, `0`, `3`. Enunciado: `2 + 2 = 4` (tipo `Prop`); prova: `meu_teorema` (tipo `2 + 2 = 4`). A subtração em $\mathbb{N}$ é "truncada" (para em 0), e `7 / 2` é o quociente 3.

Exercício 2

Dica 1

`rfl` só fecha a meta se os dois lados ficam *idênticos* depois de fazer as contas previstas nas definições. Com números concretos, dá para fazer todas as contas.

Dica 2

A adição é definida por recursão no *segundo* argumento: $n+0 = n$ e $n+(m+1) = (n+m)+1$. Em `0 + n`, com n desconhecido, qual equação se aplica?

Quase a resposta

Fecham: (a), (b), (d). Falham: (c) e (e), com uma mensagem como (pode variar um pouco no Lean Web):

```
Tactic `rfl` failed: The left-hand side
  0 + n
is not definitionally equal to the right-hand side
  n
```

(e) falha porque é *falso*: $4 \neq 5$. (c) falha mesmo sendo *verdadeiro*: `n + 0` se reduz por definição, `0 + n` não; é preciso **indução** — o assunto do Dia 2.

Exercício 3

Dica 1

Trabalhe de trás para frente: a meta é `R`. Qual hipótese *termina* em `R`?

Dica 2

`apply hqr` troca a meta `R` por `Q`. Repita a ideia. No modo termo, lembre que `hpq hp` é uma prova de `Q`.

Quase a resposta

Papel: "Por P e $P \Rightarrow Q$, vale Q; por Q e $Q \Rightarrow R$, vale R." Lean:

```
example (P Q R : Prop) (hp : P) (hpq : P → Q) (hqr : Q → R) : R := by
  apply hqr     -- ⊢ Q
  apply hpq     -- ⊢ P
```

```
  exact hp      -- No goals

example (P Q R : Prop) (hp : P) (hpq : P → Q) (hqr : Q → R) : R := hqr (hpq hp)
```

Exercício 4

Dica 1

Uma linha como `x y : ℕ` só apresenta objetos; hipóteses têm nome e uma afirmação (`h : y = x + 7`). No nível 2, `rw [h]` troca `y` por `x + 7`.

Dica 2

No $2 + 2 = 4$, a ideia é escrever tudo com `succ` e `0`, usar `add_succ` e `add_zero`, e depois voltar aos numerais. Para trocar só o *segundo* `2`, use `nth_rewrite 2 [two_eq_succ_one]`.

Quase a resposta

Uma sequência que funciona no jogo (sintaxe do NNG; os nomes podem variar entre versões):

```
nth_rewrite 2 [two_eq_succ_one]  -- 2 + succ 1 = 4
rw [add_succ]                    -- succ (2 + 1) = 4
rw [one_eq_succ_zero]            -- succ (2 + succ 0) = 4
rw [add_succ]                    -- succ (succ (2 + 0)) = 4
rw [add_zero]                    -- succ (succ 2) = 4
rw [←  three_eq_succ_two]         -- succ 3 = 4
rw [←  four_eq_succ_three]        -- 4 = 4
rfl
```

(Na sua tela, `succ` pode aparecer escrito de outro jeito; a ideia é a mesma.)

Exercício 5

Dica 1

Se a resposta tem `begin ... end` ou `nat.succ` (minúsculo), é Lean 3, que não funciona no Lean 4. Peça de novo: "em Lean 4".

Dica 2

Se o erro fala de um nome desconhecido (`unknown identifier` ou `unknown constant`), a IA inventou ou errou o nome de um lema. É uma alucinação que o Lean pegou.

Quase a resposta

(a) Uma prova correta é curta, por exemplo `example : ∀ n : ℕ, n + 0 = n := by intro n; rfl` (`intro n` = "seja n"), ou em modo termo `fun n => rfl`. (b) Uma prova correta precisa de `induction`; se a IA usou só `rfl`, o Lean recusou pelo mesmo motivo do Exercício 2(c). Se ela usou um lema pronto da Mathlib (como `Nat.zero_add`), o Lean aceita — e no Dia 2 vamos provar esse lema nós mesmos.

Exercício 6

Dica 1

(a) Releia a seção "O que o Lean *não* faz" desta folha: quem *procura* a prova em cada caso? (b) Comece com `intro h`; olhe o contexto.

Dica 2

(b) Com `h : P ∧ Q`, `h.1 : P` e `h.2 : Q`. Para provar `Q ∧ P`, entregue um par ⟨`prova de Q, prova de P`⟩.

Quase a resposta

(a) Exemplo: "Um SAT solver decide sozinho se um enunciado, numa lógica restrita, é satisfazível. O Lean trabalha numa lógica muito mais expressiva, onde isso é impossível em geral; por isso quem constrói a prova é você (ou uma IA), e o Lean só confere — com certeza total." (b)

```
example (P Q : Prop) : P ∧ Q → Q ∧ P := by
  intro h            -- h : P ∧ Q  ⊢ Q ∧ P
  exact \LANG{}h.2, h.1\RANG{}

example (P Q : Prop) : P ∧Q → Q ∧P := fun h => \LANG{}h.2, h.1\RANG{}
```